

SISTEMAS DISTRIBUIDOS

PRACTICA EVCHARGING

María Jover Espí (53977393G)

Haytam Abaran Said (74532427F)

Curso 2025-2026

Índice

1. Informe de desarrollo	2
1.1. Componentes	2
1.1.1. Central	2
1.1.2. Puntos de carga (CP)	2
1.1.3. Driver	3
1.2. Tópicos	3
1.2.1. Supply Request	3
1.2.2. Supply Response	3
1.2.3. Supply Completed	3
1.2.4. Supply Progress	3
1.2.5. Supply Direct Start	3
1.2.6. CP Status	4
1.2.7. CP Command	4
1.2.8. Driver Status	4
2. Protocolo estándar de empaquetado	5
3. Detalle de despliegue	6
3.1. Dependencias de librerías	6
3.2. Configuración Kafka	6
3.3. Despliegue de componentes	7
4. Funcionamiento de la aplicación	8

1. Informe de desarrollo

1.1. Componentes

1.1.1. Central

La central gestiona, opera y monitoriza los puntos de carga. Utiliza Kafka para comunicarse con otros componentes. Sus funcionalidades son:

- Registro y gestión de los CPs → Mantiene la información de todos los puntos de carga.
- Autorización de suministros → Valida que un punto de recarga esté disponible antes de autorizar una carga solicitada por un driver.
- Monitorización en tiempo real → Muestra en el panel web los estados, precios, Kwh y el porcentaje de progreso del suministro.
- Control de operaciones → Envía comandos a los CPs para cambiar sus estados.
- Gestión de suministros → Coordina todo el flujo desde la solicitud hasta la finalización.

1.1.2. Puntos de carga (CP)

Estaciones donde el usuario podrá cargar su vehículo eléctrico. Tenemos dos tipos que se conectarán entre sí:

- Engine → Contiene la lógica principal de todo el sistema de CP. Sus funcionalidades son:
 - Registro en la central → Se conectará y autenticará con la central.
 - Gestión de estados → Maneja los estados.
 - Ejecución de suministros → Realiza las cargas.
 - Telemetría en tiempo real → Durante el suministro, envía actualizaciones cada segundo con el consumo en kWh y el importe en euros.
 - Respuesta a comandos → Recibe y ejecuta ordenes de la central.
 - Simulación de conexión → Tiene opciones de menú para simular cuando el usuario conecte o desconecte el vehículo del CP.
 - Suministro directo → Desde el CP se puede suministrar seleccionando cualquier Driver.
- Monitor → Comprueba la seguridad del CP, enviando un mensaje con el estado de este. Sus funcionalidades son:
 - Monitorización continua → Envía cada segundo un mensaje de comprobación de estado al Engine.
 - Detección de averías → Si no recibe respuesta o recibe un KO por parte del Engine, notifica inmediatamente a la Central.
 - Autenticación y registro → Ayuda en el proceso de registrar el CP en la central.
 - Gestión de emergencias → Cuando detecta una avería, informa a la central para que actualice el estado. Si se resuelve, también lo notifica.
 - Interrupción de suministros → Si hay una avería durante un suministro, debe finalizarse inmediatamente.

1.1.3. Driver

Aplicación que dispondrán los usuarios del sistema para solicitar suministro del CP. Sus funcionalidades son:

- Solicitud de suministros → Envía peticiones de carga a la central especificando el CP deseado.
- Visualización de estado → Muestra en pantalla el estado del CP que le está suministrando energía.
- Recepción de notificaciones → Recibe todas las actualizaciones de proceso (autorización, inicio, progreso, finalización).
- Visualización del consumo → Muestra en tiempo real los kWh consumidos y el importe mientras se carga.
- Recepción del ticket final → Al terminal, recibe el comprobante con el total de la carga y crea el ticket con esa información.
- Automatización de pruebas → Puede leer un archivo con múltiples servicios y solicitarlos secuencialmente.
- Vista de disponibilidad → Puede ver todos los CPs disponibles en la red.

1.2. Tópicos

1.2.1. Supply Request

Peticiones entre el driver con la central y los CP. El driver es quien va a solicitar la energía y el CP procesará las solicitudes.

1.2.2. Supply Response

Respuestas que envían la central y los CP al driver. Informa si el suministro fue aceptado, denegado o interrumpido (manejo desde la interfaz).

1.2.3. Supply Completed

El CP indica que el suministro ha terminado.

1.2.4. Supply Progress

El CP publica las actualizaciones en tiempo real de los suministros que son monitoreadas por el driver y la central.

1.2.5. Supply Direct Start

El CP hace un suministro directo sin confirmación del Driver.

1.2.6. CP Status

Manda los estados que tienen las CP. Estos estados son los siguientes:

- Out of Order → El CP funciona correctamente, pero no podrá utilizarse hasta que la central lo active. Simplemente está parado.
- Activo → El CP funciona y espera una solicitud de recarga.
- Desconectado → El CP no está conectado al sistema central, no será posible monitorizar su uso
- Averiado → El CP está conectado a la central, pero no es posible su uso debido a que está averiado.
- Suministrando → El CP cambiara a este estado cuando este suministrando energía al vehículo del usuario. Solo podrá suministrar si el CP esta activo y cuando termine de suministrar volverá a dicho estado.

1.2.7. CP Command

La central envía las ordenes necesarias para cambiar de estado a la CP.

1.2.8. Driver Status

Escucha el estado del Driver que puede ser: libre, suministrando y desconectado (no hay Driver).

2. Protocolo estándar de empaquetado

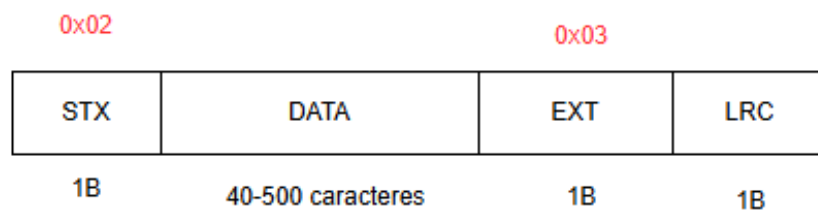
En nuestro proyecto hemos seguido el protocolo estándar <STX><DATA><ETX><LRC> que es un protocolo basado en un formato de empaquetado de mensajes, usando comunicaciones seriales y TCP.

Se divide en 4 componentes:

- STX → Marcara el comienzo del mensaje con el valor hexadecimal 0x02.
- DATA → Datos o comandos que se transmiten entre los dispositivos.
- EXT → Marcara el final del mensaje con el valor hexadecimal 0x03.
- LRC → Verificación de mensajes. Comprobara los errores mediante cálculos con operaciones XOR byte a byte.

También utilizara caracteres de control especiales para gestionar la transmisión:

- ACK → Indica que el mensaje se recibió correctamente. Su valor es de 0x06.
- NACK → Indica que hubo un error en el mensaje (comprobado por el LRC). Su valor es de 0x15.



Anatomía del Protocolo.

Hemos realizado el flujo de la siguiente manera:

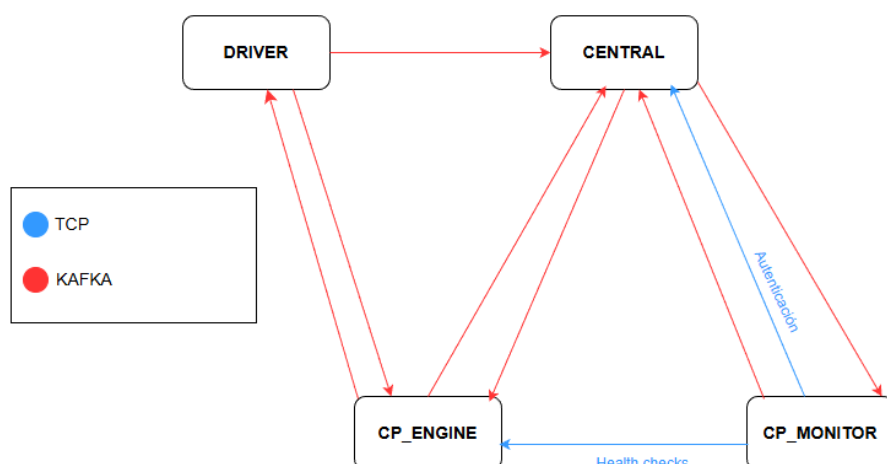


Diagrama de flujo.

3. Detalle de despliegue

3.1. Dependencias de librerías

Comenzaremos instalando algunas librerías de Python que hemos requerido para realizar la práctica. Para ello hemos hecho un fichero con las librerías necesarias:

Pip install -r requirements.txt

En el fichero requirements.txt podemos encontrar las siguientes librerías:

- Confluent_kafka → Permite producir y consumir mensajes, manejo de backpressure, comprensión y particionado avanzado para entornos de producción.
- Requests → Librería HTTP sencilla para realizar peticiones REST (GET/POST/PUT/DELETE), manejar sesiones, headers, código JSON y timeouts.
- Colorama → Facilita colorear texto y estilos en la terminal.
- Flask → Microframework web para crear APIs y aplicaciones HTTP con routing, middleware y extensiones.
- Flask_socketio → Extensión de Flask basado en el soporte Socket.io (websockets) para eventos bidireccionales entre servidor y navegador, con salas, broadcast y autenticación.
- Eventlet → Librería de concurrencia cooperativa basada en green threads y monkey patching de sockets, óptima para I/O intensivo. Se sincroniza con Flask-socketio.
- Python-socketio → Implementación de cliente/servidor con Socket.IO en Python. Permite crear servidores con varios backends o clientes que se conectan a los servidores de Socket.io
- Python-engineio → Capa base del protocolo Engine.IO en Python. Gestiona handshakes, heartbeats y transporte long-polling/websocket.
- Reportlab → Toolkit para generar PDFs. Produce documentos server-side sin depender de un navegador. Utilizado para generar los tickets.

NOTA: Para visualizar los tickets desde Visual Studio Code puedes instalar la extensión `vscode-pdf`.

3.2. Configuración Kafka

1. En primer lugar, configuraremos el archivo `server.properties` en la carpeta `config`
2. Formatearemos los logs, generando el token UUID. Para ello arrancaremos el archivo `formateo-logs.bat`.
3. Arrancaremos Kafka desde el fichero `start-kafka.bat`.
4. Después se crearán los tópicos mencionados anteriormente (punto 1.2) desde el archivo `crear-topicos.bat`.

Hay que cambiar las ips de la ejecución de los topics cada vez que cambiemos de dispositivo.

3.3. Despliegue de componentes

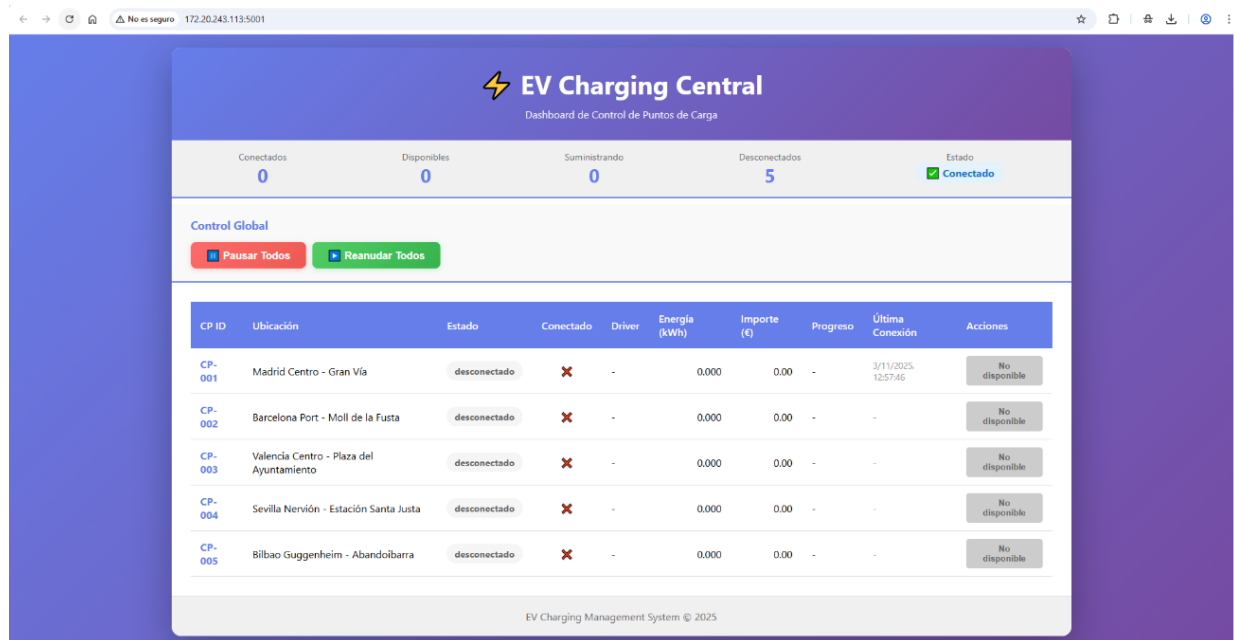
- Central:
python EV_Central.py --kafka <ip_kafka:puerto_kafka> --tcp-port <puerto_escucha> --web-port <puerto_web> --db evcharging.db
- CP Engine:
python EV_CP_E.py --cp-id <id_CP> --kafka <ip_kafka:puerto_kafka> --ubicacion <ubicación_CP> --precio-kwh <precio_kwh> --engine-port <puerto_escucha> --web-port <puerto_web> --central-web-port <puerto_protocolo>
- CP Monitor:
python EV_CP_M.py --kafka <ip_kafka:puerto_kafka> --cp-id <id_CP> --engine <ip_engine:puerto_engine>
- Driver:
python EV_Driver.py --driver-id <id_driver> --kafka <ip_kafka:puerto_kafka> --web-port <puerto_web> --services-file servicios.txt

Si quieres arrancar el driver sin servicios simplemente ignora esta parte:

--services-file servicios.txt

4. Funcionamiento de la aplicación

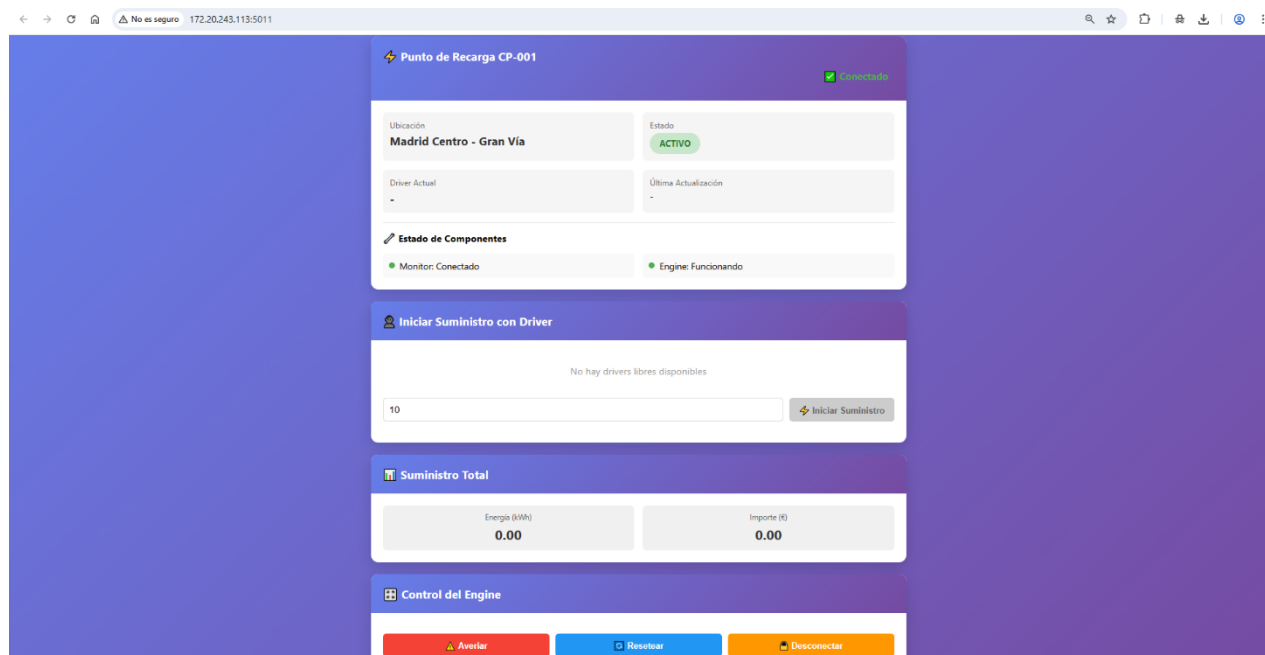
1. Iniciaremos la central mediante EV_Central.py, donde mediante su interfaz podremos observar los puntos de carga (CP) que existen en nuestra aplicación. Desde este componente podemos apagar o encender los puntos de carga, tanto individual como grupal.
Como todavía no se ha iniciado ningún CP (siguiente paso), mostraran todos en desconectado.



También podemos fijarnos en la terminal, mostrará que la central está conectada correctamente.

```
C:\Users\alumno\Desktop\EVCharging>python EV_Central.py --kafka 172.20.243.113:9092 --tcp-port 5000 --web-port 5001 --db evcharging.db
[DB] BD: evcharging.db
[CENTRAL] CPs cargados desde BD: 5
[TCP] Puerto 5000 [PROTOCOLO ESTÁNDAR][KAFKA] Escuchando CP_STATUS
[KAFKA] Escuchando SUPPLY_REQUEST[KAFKA] Escuchando SUPPLY_PROGRESS
[KAFKA] Escuchando SUPPLY_COMPLETED[KAFKA] Escuchando SUPPLY_DIRECT_START[CENTRAL] Web puerto 5001
[WEB] Cliente conectado
[WEB] Cliente conectado
```

2. Iniciaremos el CP mediante los archivos EV_Engine.py y EV_Monitor.py. En su interfaz observaremos los drivers disponibles y el estado de ese CP.
- En el CP podemos iniciar un suministro, averiar, reiniciar y desconectar el CP de la central. También mostrara el estado de dicha CP.



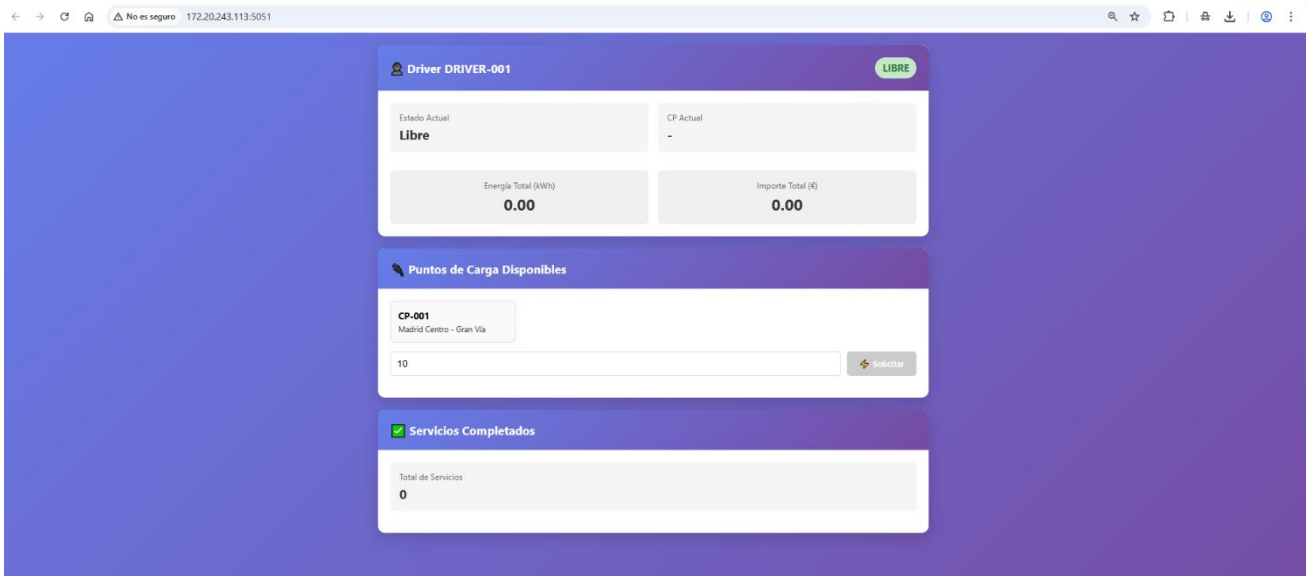
Comprobaremos en la terminal que se ha conectado correctamente:

```
C:\Users\alumno\Desktop\EVCharging>python EV_CP_E.py --cp-id CP-001 --kafka 172.20.243.113:9092 --
ubicacion "Madrid Centro" --precio-kwh 0.35 --engine-port 6000 --web-port 5011 --central-web-port
5001
[CENTRAL-SYNC] Consultando: http://172.20.243.113:5001/api/cp/CP-001/info
[CENTRAL-SYNC] Datos recibidos de Central: Madrid Centro - Gran Vía, 0.3€/kwh
[CP-INIT] De Central: Madrid Centro - Gran Vía, 0.3€/kwh
[ENGINE-TCP] Puerto 6000 [PROTOCOLO ESTÁNDAR][KAFKA-CMD] Escuchando para CP-001
[KAFKA-DRIVER] Escuchando DRIVER_STATUS[CP-ENGINE] CP CP-001 iniciando
[CP-ENGINE] Engine TCP: 6000
[CP-ENGINE] Web: 5011
[CP-ENGINE] Protocolo: ESTÁNDAR <STX><DATA><ETX><LRC>
* Serving Flask app 'EV_CP_E'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://172.0.0.1:5011
* Running on http://172.20.243.113:5011
Press CTRL+C to quit
[ENGINE-TCP] Monitor: ('172.20.243.113', 60041)
[ENGINE-TCP] Monitor conectado [PROTOCOLO OK]
[KAFKA-PUBLISH] activo
[KAFKA-STATUS] Estado actualizado: activo + activo
[KAFKA-STATUS] Estado actualizado: activo + desconectado
[KAFKA-STATUS] Estado actualizado: desconectado + activo
[KAFKA-STATUS] Estado actualizado: activo + activo

C:\Users\alumno\Desktop\EVCharging>python EV_CP_M.py --kafka 172.20.243.113:9092 --cp-id CP-001 --
engine 172.20.243.113:6000
[MONITOR] CP Monitor CP-001 iniciando
[MONITOR] Engine: 172.20.243.113:6000
[MONITOR] Kafka: 172.20.243.113:9092
[MONITOR] TCP Port: 9092
[MONITOR] Protocolo: ESTÁNDAR <STX><DATA><ETX><LRC>
[HEALTH] Usando PROTOCOLO ESTÁNDAR <STX><DATA><ETX><LRC>
[CONNECT] Intentando conectar con Engine usando PROTOCOLO ESTÁNDAR
[KAFKA] Escuchando CP_STATUS para CP-001[TCP-SERVER] Escuchando en puerto 9092 [PROTOCOLO ESTÁNDAR
][COMMAND] Escuchando CP_COMMAND para CP-001
[CONNECT] ✓ Conectado con Engine en 172.20.243.113:6000 [PROTOCOLO OK]

[PUBLISHER] Publicando estado cada 3 segundos
[HEALTH] Monitor + Engine OK + activo
[PUBLISHER] Publicado: desconectado
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
[PUBLISHER] Publicado: activo
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
[PUBLISHER] Publicado: activo
[KAFKA] CP dice: activo
[HEALTH] Monitor + Engine OK + activo
[HEALTH] Monitor + Engine OK + activo
```

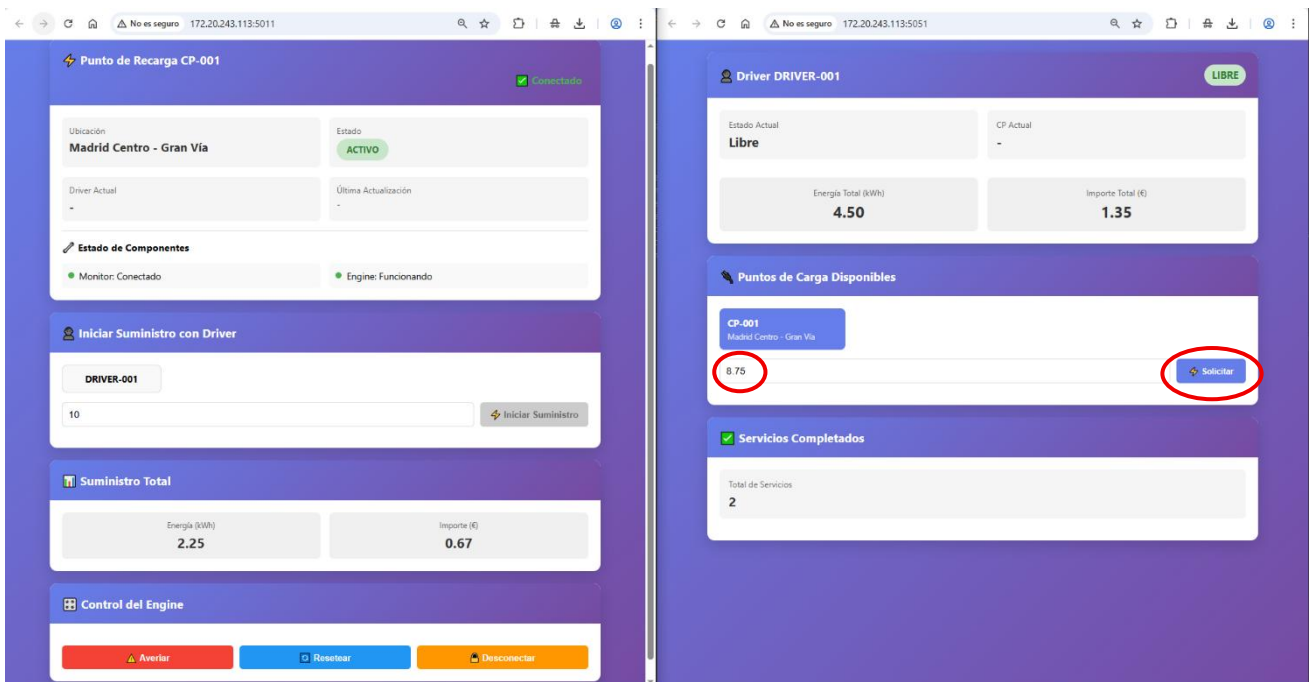
3. Iniciaremos el Driver mediante EV_Driver.py. En la interfaz del Driver podemos observar los CP disponibles. En el Driver también podremos suministrar, además de observar la cantidad de servicios que ha completado.

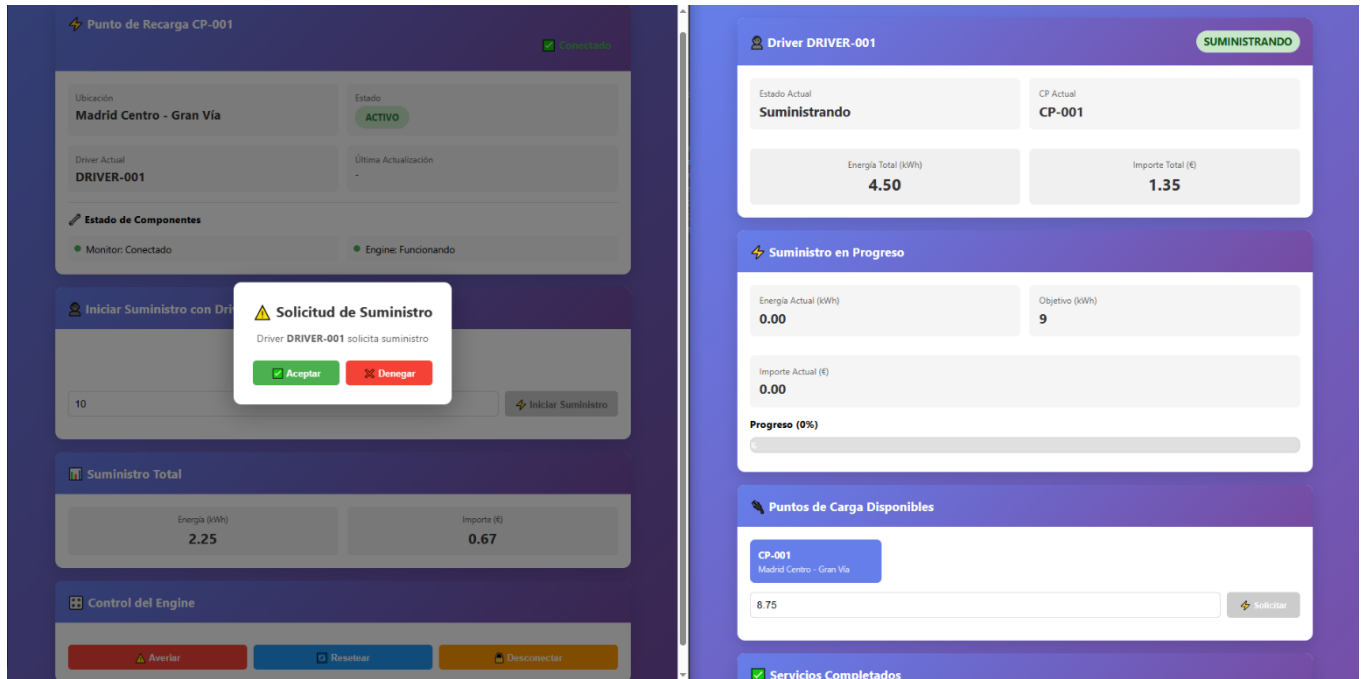


También observaremos la terminal:

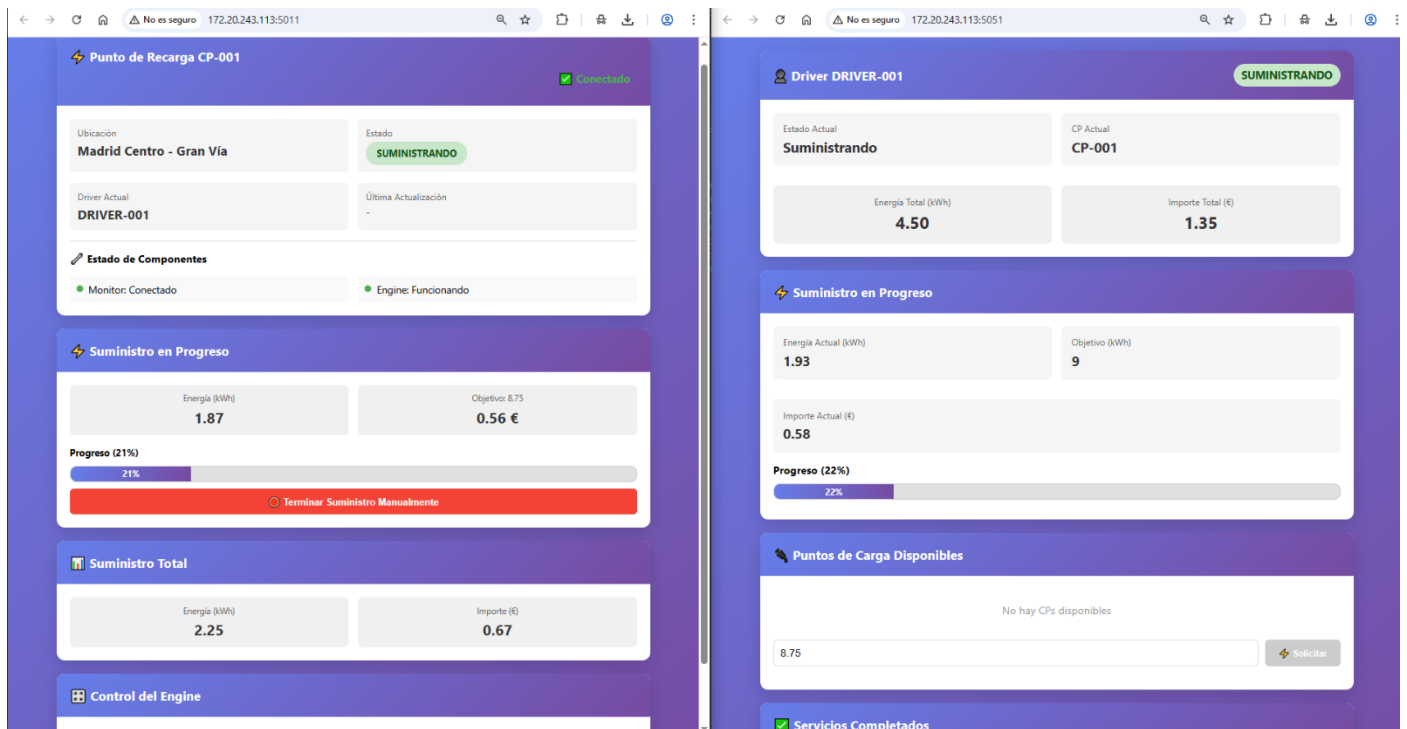
```
C:\Users\alumno\Desktop\EVCharging>python EV_Driver.py --driver-id DRIVER-001 --kafka 172.20.243.113:9092 --web-port 5051
[KAFKA] Escuchando CP_STATUS
[KAFKA] Escuchando SUPPLY_RESPONSE
[KAFKA] Escuchando SUPPLY_PROGRESS
[KAFKA] Escuchando SUPPLY_DIRECT_START
[KAFKA] Publicando estado de driver
[DRIVER] Driver DRIVER-001 iniciando
[DRIVER] Kafka: 172.20.243.113:9092
[DRIVER] Web: 5051
```

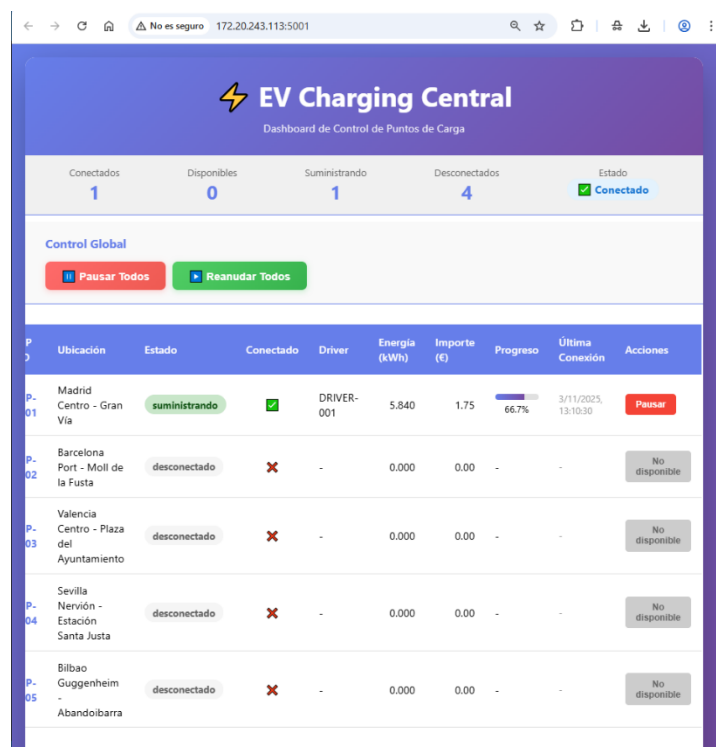
4. Realizaremos un ejemplo de suministro. Desde el Driver seleccionaremos una CP disponible y pondremos los kWh que queramos suministrar, en este caso suministraremos 8.75 kWh. Cuando el Driver lo solicite, tendrá que esperar una confirmación del CP.





5. Cuando comience el suministro, en todos los componentes (Driver, CP y Central) aparecerá una barra de progreso y también mostrara el precio y los kWh que se incrementaran cada segundo. En el CP aparecerá un botón donde podremos terminar el suministro inmediatamente.





6. Cuando termine el suministro, se generará el ticket correspondiente en formato PDF:

■ TICKET DE SUMINISTRO

CAMPO	VALOR
Driver ID	DRIVER-001
CP ID	CP-001
Energía (kWh)	8.750
Importe (€)	2.62
Fecha/Hora	03/11/2025 13:10:37
Estado	■ COMPLETADO

Generado: 03/11/2025 13:10:37 | Sistema EV-CP v1.0 | Ticket automático

El estado del ticket cambiará dependiendo de:

- Completado → Si el suministro se ha completado satisfactoriamente.
- Completado (manual) → Si el CP ha terminado el suministro mediante el botón mencionado en el punto 5.
- Averiado → Si en medio del suministro el CP se avería
- Out of order (fuera de servicio) → Si en medio del suministro el CP se ha desconectado de la central.